

IBM CERTIFIED ASSOCIATE DEVELOPER QUANTUM COMPUTATION

Deep-Dive Study Guide • Exam C1000-112

Heavy Code Edition • Qiskit v1.x • 2026

ASSOCIATE

DEVELOPER

QUANTUM

60 questions • 90 minutes • Score 43/60 to pass (72%)

Covers: Qiskit circuits, gates, algorithms, simulators, transpilation & real hardware

TABLE OF CONTENTS

- SECTION 1** — Exam Blueprint & Domain Weights
- SECTION 2** — Qiskit Installation & Environment Setup
- SECTION 3** — Quantum Circuits – QuantumCircuit API
- SECTION 4** — Gates – Single Qubit, Multi-Qubit, Parameterised
- SECTION 5** — Measurements, Classical Registers & Conditionals
- SECTION 6** — Simulators – AerSimulator, StatevectorSimulator
- SECTION 7** — Visualisation – draw(), plot_histogram(), Bloch Sphere
- SECTION 8** — Transpilation – PassManager, Optimization Levels
- SECTION 9** — Real Hardware – IBM Quantum, Job Management
- SECTION 10** — Quantum Algorithms – Deutsch, Grover, QFT, Teleportation
- SECTION 11** — Practice Exam – 40 Questions with Answers & Explanations
- SECTION 12** — Quick-Reference Cheat Sheet

SECTION 1

Exam Blueprint & Domain Weights

The C1000-112 exam tests your ability to write, execute, and analyse quantum circuits using Qiskit. 60 multiple-choice questions in 90 minutes. A score of 43+ (72%) passes. Questions are scenario-based — you will read code and predict output, or choose the correct API call.

Domain	Weight	Key Topics
Working with Qiskit	~32%	QuantumCircuit, gates, registers, compose, append
Visualising Quantum States	~17%	draw(), plot_histogram(), plot_bloch_multivector()
Running Circuits on Backends	~22%	AerSimulator, transpile(), run(), job.result()
Quantum Algorithms	~29%	Grover, QFT, Deutsch-Jozsa, quantum teleportation

Exam Strategy: 32%+29% = 61% is circuits + algorithms. Master QuantumCircuit and the 4 algorithms cold. The remaining 39% (viz + backends) is straightforward API recall. Aim for 90%+ on circuits/algorithms; 80%+ elsewhere.

SECTION 2

Installation & Environment Setup

Install the exact versions tested by the exam. Qiskit v1.x is the current exam baseline.

```
# Create isolated environment (strongly recommended)
python -m venv qiskit-env
source qiskit-env/bin/activate # Windows: qiskit-env\Scripts\activate

# Install Qiskit v1.x + Aer simulator
pip install qiskit==1.3.0
pip install qiskit-aer==0.14.2
pip install qiskit-ibm-runtime==0.26.0

# Verify
python -c "import qiskit; print(qiskit.__version__)" # 1.3.0
python -c "from qiskit_aer import AerSimulator; print('Aer OK')"
```

IBM Quantum Account Setup — required for running on real hardware:

```
from qiskit_ibm_runtime import QiskitRuntimeService

# One-time save of your API token (get it from quantum.ibm.com)
QiskitRuntimeService.save_account(
    channel='ibm_quantum',
    token='YOUR_API_TOKEN_HERE',
    overwrite=True
)

# Load in subsequent sessions
service = QiskitRuntimeService(channel='ibm_quantum')
backends = service.backends() # list available backends
print([b.name for b in backends])
```

Exam Tip: You will NOT need an IBM account for the exam. All exam questions use simulators or conceptual backend selection. But account setup is essential for hands-on practice — do it on Day 1.

SECTION 3

Quantum Circuits – QuantumCircuit API

QuantumCircuit is the central object in Qiskit. Every exam question either creates, manipulates, or queries a QuantumCircuit. Know every constructor signature and every method cold.

Construction Patterns

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

# Pattern 1: Simple integer constructor
qc1 = QuantumCircuit(2) # 2 qubits, no classical bits
qc2 = QuantumCircuit(2, 2) # 2 qubits + 2 classical bits

# Pattern 2: Named registers (important for multi-register circuits)
qr = QuantumRegister(3, name='q') # 3 qubits named q[0], q[1], q[2]
cr = ClassicalRegister(3, name='c') # 3 classical bits
qc3 = QuantumCircuit(qr, cr)

# Pattern 3: Multiple registers (exam favourite!)
qr_a = QuantumRegister(2, 'a') # a[0], a[1]
qr_b = QuantumRegister(2, 'b') # b[0], b[1]
cr = ClassicalRegister(4, 'c')
qc4 = QuantumCircuit(qr_a, qr_b, cr)

# Accessing circuit properties
print(qc4.num_qubits) # 4
print(qc4.num_clbits) # 4
print(qc4.qregs) # [QuantumRegister(2, 'a'), QuantumRegister(2, 'b')]
print(qc4.depth()) # 0 (no gates yet)
print(len(qc4)) # 0 (number of operations)
```

Circuit Composition

```
# compose() – attach one circuit to another (non-destructive, returns new circuit)
qc_a = QuantumCircuit(2)
qc_a.h(0)
qc_b = QuantumCircuit(2)
qc_b.cx(0, 1)

qc_combined = qc_a.compose(qc_b) # qc_a is unchanged
# Note: compose() aligns by qubit index by default

# compose() with qubit remapping
qc_full = QuantumCircuit(3)
sub = QuantumCircuit(2)
sub.cx(0, 1)

# Apply sub-circuit to qubits [1,2] of qc_full
qc_full.compose(sub, qubits=[1, 2], inplace=True)
```

```
# append() – add a gate/instruction to specific qubits
from qiskit.circuit.library import HGate
qc = QuantumCircuit(3)
qc.append(HGate(), [1]) # Hadamard on qubit 1

# barrier() – visual separator + prevents gate optimisation across it
qc = QuantumCircuit(3)
qc.h([0,1,2])
qc.barrier() # --- separator ---
qc.measure_all()
```

Circuit Introspection

```
qc = QuantumCircuit(3, 3)
qc.h(0); qc.cx(0,1); qc.cx(1,2); qc.measure([0,1,2],[0,1,2])

print(qc.depth()) # critical path length = 4
print(qc.count_ops()) # OrderedDict: {'h':1, 'cx':2, 'measure':3}
print(qc.num_qubits) # 3
print(qc.num_clbits) # 3
print(qc.size()) # total gate count = 6

# Iterate over circuit instructions
for instruction in qc.data:
    print(instruction.operation.name,
          [q._index for q in instruction.qubits])
# Output:
# h [0]
# cx [0, 1]
# cx [1, 2]
# measure [0] ...
```

EXAM WATCH: *depth()* counts the longest path through the circuit, NOT the total number of gates. A circuit with 3 parallel H gates has *depth*=1, *size*=3. This distinction appears in ~3 exam questions.

SECTION 4

Gates – Single, Multi-Qubit & Parameterised

Single-Qubit Gates

```
from qiskit import QuantumCircuit
import numpy as np

qc = QuantumCircuit(1)

# Pauli gates
qc.x(0) # NOT gate: |0> -> |1>, |1> -> |0>
qc.y(0) # Y gate: adds phase flip
qc.z(0) # Phase flip: |1> -> -|1>

# Hadamard – creates superposition
qc.h(0) # |0> -> (|0>+|1>)/sqrt(2)

# Phase gates
qc.s(0) # S gate: Z^(1/2), phase = pi/2
qc.sdg(0) # S-dagger: inverse of S
qc.t(0) # T gate: Z^(1/4), phase = pi/4
qc.tdg(0) # T-dagger

# Rotation gates (parameterised)
qc.rx(np.pi/4, 0) # Rotate around X-axis by pi/4
qc.ry(np.pi/3, 0) # Rotate around Y-axis by pi/3
qc.rz(np.pi/2, 0) # Rotate around Z-axis by pi/2

# Universal single-qubit gate
qc.u(np.pi/2, 0, np.pi, 0) # U(theta, phi, lambda)
# Equivalent to H gate when theta=pi/2, phi=0, lambda=pi

# Apply gate to ALL qubits
qc2 = QuantumCircuit(4)
qc2.h([0, 1, 2, 3]) # H on all 4 qubits
qc2.x(range(4)) # X on all 4 qubits
```

Multi-Qubit Gates

```
qc = QuantumCircuit(3)

# CNOT (CX) – most important 2-qubit gate
qc.cx(0, 1) # control=0, target=1
qc.cx(1, 2) # control=1, target=2

# CZ – controlled-Z
qc.cz(0, 1) # flips phase of |11> state

# SWAP – exchange qubit states
qc.swap(0, 2) # swaps q[0] and q[2]
```

```
# Toffoli (CCX) – 3-qubit, flips target if both controls are |1>
qc.ccx(0, 1, 2) # controls=0,1; target=2

# Controlled versions of arbitrary gates
qc.ch(0, 1) # controlled-H
qc.crx(np.pi/2, 0, 1) # controlled-Rx
qc.cu(np.pi/2, 0, np.pi, 0, 0, 1) # controlled-U

# Build Bell state (maximally entangled pair)
bell = QuantumCircuit(2, 2)
bell.h(0) # superposition on qubit 0
bell.cx(0, 1) # entangle qubit 0 and 1
bell.measure([0,1], [0,1])

# Statevector: (|00> + |11>) / sqrt(2)
# Measurement: always 00 or 11, never 01 or 10
```

Parameterised Circuits

```
from qiskit.circuit import Parameter, ParameterVector
import numpy as np

# Single parameter
theta = Parameter('theta')
qc = QuantumCircuit(1)
qc.ry(theta, 0)
print(qc.parameters) # ParameterView({theta})

# Bind a value
bound_qc = qc.assign_parameters({theta: np.pi/3})

# ParameterVector – for variational circuits (VQE, QAOA)
params = ParameterVector('p', length=6) # p[0]..p[5]
qc_var = QuantumCircuit(3)
for i in range(3):
    qc_var.ry(params[i], i) # rotation layer
for i in range(3):
    qc_var.rz(params[i+3], i) # phase layer
qc_var.cx(0, 1); qc_var.cx(1, 2) # entanglement

# Bind all parameters at once
import numpy as np
vals = np.random.uniform(0, 2*np.pi, 6)
bound = qc_var.assign_parameters(dict(zip(params, vals)))

# Bind as a list (order matches ParameterView sorted order)
bound2 = qc_var.assign_parameters(vals)
```

EXAM WATCH: `assign_parameters()` does NOT modify the original circuit. It returns a new circuit. Questions often test whether you know the original remains unbound after calling `assign_parameters()`.

SECTION 5

Measurements, Classical Registers & Conditionals

```

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

# Method 1: measure(qubit, clbit) – explicit mapping
qc = QuantumCircuit(3, 3)
qc.h(0); qc.cx(0,1); qc.cx(1,2)
qc.measure(0, 0) # qubit 0 -> classical bit 0
qc.measure(1, 1) # qubit 1 -> classical bit 1
qc.measure(2, 2) # qubit 2 -> classical bit 2

# Method 2: measure with lists (most common)
qc.measure([0,1,2], [0,1,2]) # same as above

# Method 3: measure_all() – adds classical register automatically
qc2 = QuantumCircuit(3)
qc2.h([0,1,2])
qc2.measure_all() # adds 'meas' ClassicalRegister(3)

# Classical conditionals (mid-circuit measurement)
qc3 = QuantumCircuit(2, 1)
qc3.h(0)
qc3.measure(0, 0)
# Apply X to qubit 1 IF classical bit 0 == 1
with qc3.if_test((qc3.clbits[0], 1)):
    qc3.x(1)

# Reading measurement results
from qiskit_aer import AerSimulator
from qiskit import transpile

sim = AerSimulator()
qc = QuantumCircuit(2, 2)
qc.h(0); qc.cx(0,1); qc.measure_all()
tqc = transpile(qc, sim)
job = sim.run(tqc, shots=1024)
result = job.result()
counts = result.get_counts() # {'00': 512, '11': 512} (approx)
print(counts)

# IMPORTANT: bit string order is REVERSED in Qiskit
# counts key '01' means: qubit 0 = 1, qubit 1 = 0 (rightmost = q[0])
# This is the #1 source of confusion on the exam!

```

CRITICAL EXAM FACT: Qiskit bit string ordering is right-to-left. In counts key '10', the rightmost '0' is qubit 0, leftmost '1' is qubit 1. So '10' means $q[0]=0$, $q[1]=1$. This appears in 4–6 exam questions.

SECTION 6

Simulators – AerSimulator & Statevector

[illegible]

SECTION 7

Visualisation

[illegible]

EXAM WATCH: `draw('text')` output is commonly shown in questions. Know that `q_0` is the TOP wire in `text/mpl` drawing, and that the rightmost bit in the measurement result corresponds to `q_0`.

SECTION 8

Transpilation – PassManager & Optimization Levels

Transpilation converts your logical circuit into a physical circuit that can run on a specific backend — mapping qubits to physical qubits, replacing gates with native basis gates, and inserting SWAP gates where needed.

[illegible]

Exam Tip: optimization_level=3 produces the best circuit but takes longest to compile. The exam will ask you to identify which level minimises gate count (3) vs compilation time (0).

SECTION 9

Real Hardware – IBM Quantum & Job Management

[illegible]

Exam Tip: The exam tests job lifecycle — QUEUED -> RUNNING -> DONE. Know that job.result() BLOCKS. Know that job_id() lets you retrieve results in a future session. Know that least_busy() is the recommended way to select a backend.

SECTION 10

Quantum Algorithms – Full Implementations

Algorithm 1: Deutsch-Jozsa

```
# Determines if  $f:\{0,1\}^n \rightarrow \{0,1\}$  is CONSTANT or BALANCED in 1 query
# Classical: needs  $2^{n-1}+1$  queries worst case

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

def deutsch_jozsa(n, oracle_type='balanced'):
    """n = number of input qubits"""
    qc = QuantumCircuit(n+1, n) # n input qubits + 1 ancilla

    # Step 1: Initialise ancilla to  $|1\rangle$ 
    qc.x(n)

    # Step 2: Hadamard on ALL qubits (input + ancilla)
    qc.h(range(n+1))
    qc.barrier()

    # Step 3: Apply oracle
    if oracle_type == 'constant_0':
        pass # do nothing ( $f(x) = 0$  for all  $x$ )
    elif oracle_type == 'constant_1':
        qc.x(n) # flip ancilla ( $f(x) = 1$  for all  $x$ )
    elif oracle_type == 'balanced':
        # CNOT from each input qubit to ancilla
        for i in range(n):
            qc.cx(i, n)
        qc.barrier()

    # Step 4: Hadamard on input qubits only
    qc.h(range(n))

    # Step 5: Measure input qubits
    qc.measure(range(n), range(n))
    return qc

sim = AerSimulator()

# Constant oracle: measure all zeros -> '000'
qc_const = deutsch_jozsa(3, 'constant_0')
counts_c = sim.run(transpile(qc_const, sim), shots=1024).result().get_counts()
print('Constant:', counts_c) # {'000': 1024}

# Balanced oracle: measure non-zero -> '111'
qc_bal = deutsch_jozsa(3, 'balanced')
counts_b = sim.run(transpile(qc_bal, sim), shots=1024).result().get_counts()
```

```
print('Balanced:', counts_b) # {'111': 1024}

# KEY: if all-zeros -> CONSTANT; otherwise -> BALANCED
```

Algorithm 2: Grover's Search

```
# Finds a marked element in unsorted database of N items
# Speedup:  $O(\sqrt{N})$  vs  $O(N)$  classical

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
import numpy as np

def grover_oracle(n, marked):
    """Oracle that flips phase of marked state"""
    oracle = QuantumCircuit(n)
    # Flip qubits where marked state has '0'
    for i, bit in enumerate(reversed(f'{marked:0{n}b}')):
        if bit == '0':
            oracle.x(i)
    # Multi-controlled Z (phase flip on marked state)
    oracle.h(n-1)
    oracle.mcx(list(range(n-1)), n-1) # multi-controlled X
    oracle.h(n-1)
    # Unflip
    for i, bit in enumerate(reversed(f'{marked:0{n}b}')):
        if bit == '0':
            oracle.x(i)
    return oracle

def diffusion_operator(n):
    """Grover diffusion:  $2|s\rangle\langle s| - I$ """
    diff = QuantumCircuit(n)
    diff.h(range(n))
    diff.x(range(n))
    diff.h(n-1)
    diff.mcx(list(range(n-1)), n-1)
    diff.h(n-1)
    diff.x(range(n))
    diff.h(range(n))
    return diff

def grover(n, marked, iterations=None):
    if iterations is None:
        iterations = int(np.pi/4 * np.sqrt(2*n)) # optimal
    qc = QuantumCircuit(n, n)
    qc.h(range(n)) # uniform superposition
```

```

oracle = grover_oracle(n, marked)
diff = diffusion_operator(n)
for _ in range(iterations):
    qc.compose(oracle, inplace=True)
    qc.compose(diff, inplace=True)
    qc.measure(range(n), range(n))
return qc

# Search for item 6 ('110') in 3-qubit space (8 items)
n, target = 3, 6
qc = grover(n, target)
sim = AerSimulator()
counts = sim.run(transpile(qc, sim), shots=2048).result().get_counts()
top = max(counts, key=counts.get)
print(f'Found: {top} = {int(top,2)}') # Found: 110 = 6

```

Algorithm 3: Quantum Teleportation

```

# Transmit unknown qubit state |psi> using 2 classical bits + 1 ebit
# Demonstrates: entanglement as a resource

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit_aer import AerSimulator
from qiskit import transpile

def teleportation_circuit(state_prep_angle=None):
    qr = QuantumRegister(3, 'q') # q[0]=Alice's qubit, q[1]=Bell pair, q[2]=Bob
    cr = ClassicalRegister(2, 'c')
    qc = QuantumCircuit(qr, cr)

    # Step 1: Prepare Alice's state to teleport
    if state_prep_angle:
        qc.ry(state_prep_angle, 0) # arbitrary state
    else:
        qc.h(0) # teleport |+> state
        qc.barrier(label='State Prep')

    # Step 2: Create Bell pair between q[1] and q[2]
    qc.h(1)
    qc.cx(1, 2)
    qc.barrier(label='Bell Pair')

    # Step 3: Alice's Bell measurement
    qc.cx(0, 1) # entangle Alice's qubit with her Bell qubit
    qc.h(0)
    qc.measure(0, 0) # measure Alice's qubit -> c[0]
    qc.measure(1, 1) # measure Bell qubit -> c[1]
    qc.barrier(label='Alice Measurement')

```

```

# Step 4: Bob applies correction based on classical bits
with qc.if_test((cr[1], 1)): # if c[1] == 1
    qc.x(2) # apply X correction
with qc.if_test((cr[0], 1)): # if c[0] == 1
    qc.z(2) # apply Z correction

return qc

qc = teleportation_circuit(state_prep_angle=1.2)
print(qc.draw('text'))

# To verify: add state tomography on Bob's qubit (q[2])
# Expected: Bob's qubit ends in ry(1.2)|0> = same state Alice started with

```

Algorithm 4: Quantum Fourier Transform

```

# QFT: quantum analogue of DFT. Runs in  $O(n^2)$  vs  $O(n \cdot 2^n)$  classically.
# Foundation of Shor's algorithm and phase estimation.

from qiskit import QuantumCircuit
import numpy as np

def qft(n):
    """Build QFT circuit for n qubits"""
    qc = QuantumCircuit(n)
    for j in range(n-1, -1, -1): # top qubit to bottom
        qc.h(j) # Hadamard
        for k in range(j-1, -1, -1):
            angle = np.pi / (2 ** (j-k))
            qc.cp(angle, k, j) # controlled-phase
        # Swap qubits to fix bit-reversal
    for i in range(n//2):
        qc.swap(i, n-i-1)
    return qc

def iqft(n):
    """Inverse QFT = QFT dagger"""
    return qft(n).inverse()

# 4-qubit QFT
qft4 = qft(4)
print(qft4.draw('text'))
print(f'Depth: {qft4.depth()}')
print(f'Gates: {qft4.count_ops()}')

# Verify: QFT followed by IQFT = identity
from qiskit.quantum_info import Operator
qft_full = qft(3).compose(iqft(3))
op = Operator(qft_full)
print(op.is_unitary()) # True

```

```
# op.data should be ~identity matrix
```

Exam Tip: You will not be asked to derive QFT from scratch. You WILL be asked: What is the gate complexity? $O(n^2)$. What is QFT used for? Phase estimation, Shor's algorithm. What does it do? Maps computational basis to Fourier basis.

SECTION 11

Practice Exam – 40 Questions with Explanations

Work through these questions under exam conditions: 60 minutes for 40 questions. Then review every explanation — especially questions you got right by guessing.

Q1. What does `QuantumCircuit(3, 2)` create?

- A) 3 classical bits and 2 qubits
- B) 3 qubits and 2 classical bits
- C) A 3x2 quantum register
- D) 3 qubits and no classical bits

Answer: B — `QuantumCircuit(n_qubits, n_clbits)`. First argument is always qubits.

Q2. After `qc.h(0)`; `qc.cx(0,1)` on $|00\rangle$, the statevector is:

- A) $|00\rangle + |01\rangle$
- B) $(|00\rangle + |11\rangle) / \sqrt{2}$
- C) $(|01\rangle + |10\rangle) / \sqrt{2}$
- D) $|11\rangle$

Answer: B — `H` creates superposition on q_0 , `CNOT` entangles. Result is Bell state Φ^+ .

Q3. `qc.measure([0,1],[1,0])` maps:

- A) qubit 0 -> cbit 0, qubit 1 -> cbit 1
- B) qubit 0 -> cbit 1, qubit 1 -> cbit 0
- C) cbit 0 -> qubit 1, cbit 1 -> qubit 0
- D) This is invalid syntax

Answer: B — `measure(qubits_list, cbits_list)` maps positionally: $q[0] \rightarrow c[1]$, $q[1] \rightarrow c[0]$.

Q4. What is the depth of: `qc.h(0)`; `qc.h(1)`; `qc.cx(0,1)`?

- A) 1
- B) 2
- C) 3
- D) 4

Answer: B — `H(0)` and `H(1)` execute in parallel (depth=1). `CX` follows (depth=2). Total=2.

Q5. Which Aer method gives the EXACT quantum state without measurement?

- A) `AerSimulator(method='qasm')`
- B) `AerSimulator(method='statevector')`
- C) `AerSimulator(method='unitary')`
- D) `AerSimulator(method='density_matrix')`

Answer: B — `method='statevector'` gives exact amplitudes. 'qasm' uses shots.

Q6. In counts {'01': 512, '10': 512}, what is the state of qubit 0?

- A) Always $|0\rangle$
- B) Always $|1\rangle$
- C) 50% $|0\rangle$, 50% $|1\rangle$
- D) Superposition

Answer: C — Rightmost bit = $q[0]$. '01' -> $q_0=1$; '10' -> $q_0=0$. So q_0 is 50/50.

Q7. `assign_parameters()` on a `ParameterVector` circuit:

- A) Modifies the circuit in place
- B) Returns a new bound circuit; original unchanged
- C) Raises an error if any parameter is unbound
- D) Requires all parameters to be floats

Answer: B — `assign_parameters()` is non-destructive by default. Returns new circuit.

Q8. What does `transpile(qc, backend, optimization_level=3)` do?

- A) Runs the circuit on the backend
- B) Converts the circuit to backend's native gates with maximum optimisation
- C) Simulates with 3 rounds of error correction
- D) Applies 3 transpilation passes only

Answer: B — `optimization_level` controls gate merging/cancellation aggressiveness (0-3).

Q9. The Toffoli gate (CCX) flips the target qubit when:

- A) Either control qubit is $|1\rangle$
- B) The first control qubit is $|1\rangle$
- C) Both control qubits are $|1\rangle$
- D) The target qubit is $|0\rangle$

Answer: C — CCX = AND gate. Flips target only when BOTH controls are $|1\rangle$.

Q10. `qc.barrier()` primarily serves to:

- A) Measure all qubits simultaneously
- B) Reset all qubits to $|0\rangle$
- C) Prevent gate reordering during transpilation and visually separate sections
- D) Add a time delay in the circuit

Answer: C — Barriers are optimisation fences + visual separators. No physical operation.

Q11. What is the output of `qc.count_ops()` for a circuit with 2 H gates and 1 CX?

- A) 3
- B) {'h': 2, 'cx': 1}
- C) {'count': 3}
- D) [H, H, CX]

Answer: B — `count_ops()` returns an `OrderedDict` mapping gate name to count.

Q12. To run a circuit on IBM real hardware with Runtime v2, you use:

- A) `qc.run(backend)`
- B) `backend.run(qc)`
- C) `SamplerV2(backend).run([tqc])`
- D) `execute(qc, backend)`

Answer: C — Qiskit Runtime v2 uses SamplerV2 or EstimatorV2 primitives.

Q13. Grover's algorithm provides which speedup over classical search?

- A) Exponential
- B) Quadratic ($\sqrt{N}$ vs N)
- C) Linear
- D) Logarithmic

Answer: B — Grover: $O(\sqrt{N})$ queries. Classical unstructured search: $O(N)$.

Q14. `plot_bloch_multivector()` is called with:

- A) A counts dictionary
- B) A QuantumCircuit
- C) A Statevector or DensityMatrix
- D) A backend object

Answer: C — It visualises quantum state. Input must be a state object, not raw circuit.

Q15. Which gate is self-inverse (gate applied twice = identity)?

- A) S gate
- B) T gate
- C) H gate
- D) $R_z(\pi/4)$ gate

Answer: C — $H^2 = I$. X, Y, Z, CNOT are also self-inverse. $S^2 = Z$, $T^2 = S$ (not identity).

Q16. A `ParameterVector('theta', 4)` creates:

- A) 4 independent parameters named `theta_0..theta_3`
- B) One parameter named `theta` with value 4
- C) A 4-dimensional array parameter
- D) 4 qubits named `theta`

Answer: A — ParameterVector creates an indexed collection: `theta[0]`, `theta[1]`, ..., `theta[3]`.

Q17. What does `service.least_busy(min_num_qubits=5)` return?

- A) The backend with fewest total qubits
- B) The operational backend with shortest queue and at least 5 qubits
- C) The fastest backend available
- D) A local simulator with 5 qubits

Answer: B — `least_busy()` selects operational backend with fewest pending jobs.

Q18. After `qc.measure_all()`, the circuit has:

- A) Measured each qubit into a pre-existing classical register
- B) Added a new ClassicalRegister named 'meas' and measured all qubits
- C) Collapsed the quantum state permanently
- D) Applied a global measurement gate

Answer: B — `measure_all()` automatically adds a 'meas' register with n bits.

Q19. The parameter shift rule computes gradients by:

- A) Finite differences: $[f(x+\text{eps}) - f(x)]/\text{eps}$
- B) Evaluating circuit at $\theta + \pi/2$ and $\theta - \pi/2$
- C) Automatic differentiation through the gate matrix
- D) Approximating the Jacobian numerically

Answer: B — Parameter shift: $\text{grad} = [f(\theta + \pi/2) - f(\theta - \pi/2)] / 2$. Exact, not approximate.

Q20. `compose(other, qubits=[2,3])` maps:

- A) Physical qubits 2,3 of the device
- B) The qubits of 'other' onto qubits 2,3 of the base circuit
- C) Qubits 2,3 of the base circuit to qubit 0,1 of other
- D) Both A and C

Answer: C — qubits parameter specifies which qubits in base circuit the subcircuit maps TO.

Continue practising: IBM Quantum Learning has official practice assessments at learning.quantum.ibm.com. Complete all 'Basics of Quantum Information' and 'Fundamentals of Quantum Algorithms' learning paths before exam day.

SECTION 12

Quick-Reference Cheat Sheet

API / Class	Method	Description
QuantumCircuit	qc.h(q)	Hadamard gate on qubit q
QuantumCircuit	qc.cx(ctrl,tgt)	CNOT: control, target
QuantumCircuit	qc.measure(q,c)	Measure qubit q into cbit c
QuantumCircuit	qc.measure_all()	Measure all, add 'meas' register
QuantumCircuit	qc.compose(other)	Attach other circuit, return new
QuantumCircuit	qc.depth()	Critical path length
QuantumCircuit	qc.count_ops()	OrderedDict of gate counts
QuantumCircuit	qc.assign_parameters(d)	Bind params, return new circuit
AerSimulator	sim.run(tqc, shots=N)	Submit job to simulator
Job	job.result()	Block until done, get result
Result	.get_counts()	Dict of bitstring:count
Result	.get_statevector()	Numpy array of amplitudes
Statevector	Statevector(qc)	Exact state from circuit
Statevector	.probabilities_dict()	Dict of state:probability
transpile()	optimization_level=0..3	0=fast compile, 3=best circuit
QiskitRuntimeService	.least_busy()	Backend with shortest queue
SamplerV2	.run([tqc], shots=N)	Run on real backend (Runtime v2)

Bit String Order (MEMORISE): Qiskit counts keys are RIGHT-TO-LEFT. Key '01' = q[0] measured 1, q[1] measured 0. Key '10' = q[0] measured 0, q[1] measured 1.

You are ready. Study this guide, run every code block, and complete all IBM Quantum Learning paths. Register at <https://www.ibm.com/training/certification/C1000112> — good luck!